

Context: I am including my Case 2 code for the University of Chicago (Midwest) Trading Competition, 2026, which is noted on my resume.

Introduction

Even as a child, playing in a kindergarten playground, you probably remember looking up to the sky and thinking: how do allocators construct a portfolio to maximize risk-adjusted returns? Fast-forward several years, and you now have a chance to explore this question.

In this case, you are a portfolio analyst tasked with allocating a fund across 25 assets grouped into five sectors. To aid you, your company has provided you with five years of historical data for each asset. Your goal is to develop an algorithm to construct and rebalance a portfolio aimed at maximizing returns while simultaneously minimizing returns variance.

You will be given a CSV with intraday prices (30 ticks per day). The algorithm you submit will be evaluated on the twelve months immediately after the period in the CSV.

Education

SHARPE RATIO

Making money is good, but making it consistently is even better. The “Sharpe Ratio” attempts to capture this distinction by measuring how much return a portfolio earns per unit risk, and is given by

$$\text{Sharpe} = \sqrt{252} \cdot \frac{\text{mean}(r_t)}{\text{std}(r_t)}$$

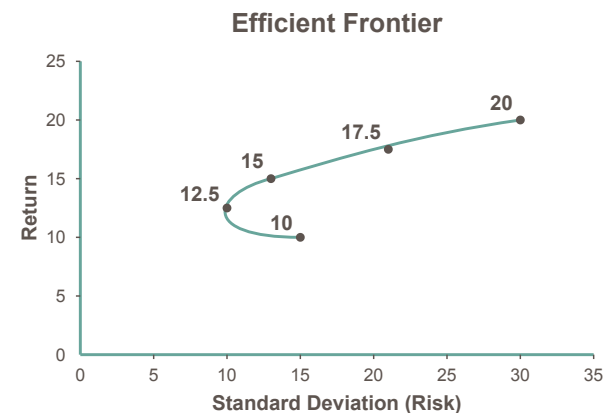
where r_t denotes the daily returns. We use the Sharpe ratio of your algorithm to compute your final score.

EQUAL WEIGHT PORTFOLIO

The naive portfolio allocation is giving each stock in your portfolio a weighting of $1/n$, assuming n distinct assets. This method assumes that your portfolio’s assets aren’t correlated with one another, and has many downsides. However, it’s often a good baseline: if your approach is underperforming the equal weight portfolio, something’s probably gone wrong. Our starter code implements this approach for you.

MARKOWITZ

In 1952, Harry Markowitz of the University of Chicago published “Portfolio Selection,” which formalized this basic intuition and formed the basis of modern portfolio theory (MPT). Markowitz conceived of an efficient frontier of portfolios that maximize return given a certain level of risk (or alternatively minimize risk given a certain desired return). Along this efficient frontier, the actual portfolio chosen is based on the individual investor’s level of risk aversion (high risk aversion = low level of risk, and vice versa). In the absence of exact knowledge on risk aversion, portfolio managers can choose an efficient frontier portfolio based on the specific objectives given to them by their clients or firms.



To implement a Markowitz portfolio, investors need some estimate of expected return and an estimate of risk. Additionally, we need some way to estimate the correlations, or covariances, between the returns of the different assets. Intuitively, a portfolio where all asset returns are highly correlated should have more risk than one with the same weights and risks for each individual asset but where all the asset returns are uncorrelated due to the fact that in the former case, one asset losing a large portion of its value means the entire portfolio likely will while this is not true in the latter case. Indeed, we see that the variance of returns in a portfolio (a measure of risk) is given by

$$\sigma_p^2 = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \text{Cov}(r_i, r_j)$$

where w_i refers to the weight of the i^{th} asset in the portfolio, r_i refers to the return of that asset, and n refers to the number of assets in the portfolio. While we can reasonably estimate variances and covariances of asset returns based on historical data, empirically, historical returns have not been a strong predictor of future expected return and large covariance matrix estimates tend to be numerically unstable, leading to practical difficulties in implementing a Markowitz portfolio. Since the development of MPT, investors and researchers have looked for new ways of dealing with this problem.

A downside of the Markowitz approach is that it works best on longer time horizons. Intraday price movements are often incredibly volatile and unpredictable— Markowitz based implementations are best used for reallocating at a 'macro' level rather than trading on a daily level.

RISK PARITY ALLOCATION (RPA)

One approach to portfolio allocation is to simply ignore expected returns when determining how to allocate assets in a portfolio. The most naive way of doing this would be to simply find the portfolio with the lowest predicted risk, but this would simply be to invest in a risk free asset, and the return offered by such a portfolio would not entice many investors. What we want instead is a way to have a portfolio full of risky assets where more weight is given to those with lower risk. This can be done by ensuring that every asset's risk contribution to the portfolio is equal, meaning that less risky assets have more weight than riskier ones in order to equalize the risk contribution. Here, risk contribution is calculated as:

$$\frac{w_i(\Sigma w)_i}{\sqrt{w^T \Sigma w}}$$

where w is the column vector of weights, and Σ is the variance-covariance matrix from the previous equation (not a summation of w). In this calculation, assets that are uncorrelated with the rest of the portfolio contribute less to risk than correlated assets with the same risk level, which is intuitive since a drop in the value of an uncorrelated asset does not contribute as much to overall losses for the portfolio as would a correlated asset. As it has been shown that asset volatilities and correlations are relatively stable over time, historical risk contribution can be used as a reliable estimate for risk contribution.

TOWARDS RETURN PREDICTION

Portfolio construction determines how to translate expected returns into weights. But where do the expected returns come from? In the simplest approaches, they are estimated from historical averages, which are insensitive to underlying structural patterns common in financial data. More sophisticated strategies attempt to forecast returns dynamically using these patterns.

Asset returns often exhibit structure that makes this possible: momentum effects, mean reversion, correlations across related assets, and clustering of volatility are all phenomena that researchers have documented and exploited. In this case, the sector structure of the universe is designed to reflect real economic relationships that influence how assets co-move and how they diverge. Signals that exploit patterns within and across sectors, or that identify periods when past returns are predictive of future ones, can compound into meaningful edges even when individual forecasts are weak. The most successful teams will go beyond static allocation and find ways to exploit predictable structure in the data.

| Case Specification & Rules

The exchange trading platform will not be used for this case. Teams are expected to develop their strategies using our Python stub code and submit their code before the competition.

You will receive historical tick price data, a sector label, bid-ask spread (in basis points), and a borrow cost (annualized cost of holding a short position, in basis points) for all 25 assets. On each trading day, your algorithm will receive the price history observed so far and must submit an allocation. These allocations will be in the form of weights on each asset: weights can be positive, negative, or zero, but the sum of their absolute values must be at most one. If your algorithm outputs weights that violate this constraint, the evaluator will rescale them proportionally. Our starter code includes an example of how your code will be evaluated; this is being provided to help you understand the evaluation process only, and should NOT be taken as predictive of your final score – ignore this at your own peril!

Evaluation uses the full intraday price path rather than only daily closing prices. For each tick, compute the portfolio return by summing, across all 25 assets, each portfolio weight multiplied by that asset's simple return derived from its log return. Short positions also pay a borrow charge each tick equal to the sum, across assets, of the short exposure times the asset's annualized borrow cost times the fraction of a year represented by one tick. At the end of each day, when the portfolio is rebalanced from old weights to new weights, transaction costs depend on the change in each asset's weight and its bid-ask spread. The total cost is the sum of two parts: a linear term equal to half the spread times the absolute weight change, and a quadratic term equal to 2.5 times the spread times the squared weight change. Our starter code includes a reference evaluator so you can reproduce these mechanics locally.

You may use any packages, programming languages, and agents to study the training data we provide, but the submitted portfolio code must be in Python and will be restricted in dependencies. The default environment used to run submitted code will use Python 3.12 and will only have NumPy, pandas, scikit-learn, and SciPy installed. More details will be published on Ed and you will have the opportunity to request for additional packages.

We strongly advise that you test your submission using a similar environment on your local machine before submitting your final code; submitted code that does not compile or that fails to run for any time step will be disqualified for this case and the team that submitted it will receive 0 points. Before final submission, there will be an opportunity to test if your code compiles and runs properly (note that the Sharpes yielded from this test will not be indicative of what your actual Sharpe ratio will be).

| Scoring

Teams will be ranked based on their annual Sharpe ratio over the 12-month test period.

| Case Materials & Data

Python stub code and training data will be released along with the case packet and additional supplementary resources through the UChicago Trading Competition Ed.

We are requiring the final code for this case to be submitted by **11:59 PM CST on Thursday, April 9th**. Note that this is different from Case 1, as we will be computing the results of this round prior to the competition. Code submitted past this deadline will not be accepted, and we reserve the right to disqualify any competitors who submit incomplete code or miss this deadline.

| Miscellaneous Tips

- Analyze returns, not prices. Prices of stocks tend to be non-stationary processes, but returns are generally stationary. Analyzing returns series will be more fruitful for your strategies than analyzing price series.
- Don't test strategies on the same data you train them on. Strategies will likely perform well on data your model has already seen – what's relevant is how well the strategy performs on data the model has not yet seen. You should not necessarily expect that your strategy will perform as well out-of-sample as it will in-sample; holding out a portion of your training data to test on (or running any other procedure to test on new data) is strongly advisable to get a more accurate sense of how successful your strategy will be. Every year, people overfit to the data – don't let that be you!
- Daily movement and intraday price changes are often two very different processes. Make sure you understand what the dynamics of each of these are—portfolio optimization is fundamentally about trading off short-term volatility with long-term predictability.
- Think carefully about costs. The scoring uses net returns after transaction costs and borrow fees. A strategy that rebalances frequently needs to outperform a lower-frequency one to justify the additional cost.

| Questions

For questions regarding Case 2, please post in the UChicago Trading Competition Ed with the "Portfolio Optimization" tag.